An Industrial Training Report

on

DIYA: DESIGN AND DEVELOPMENT OF AN UNATTENDED MEDITATION KIOSK

*carried out as part of the **Industrial Training ([Subject Code])** Submitted*

by

Ayush Gupta
23FE10CDS00349

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY (B.Tech)

in

Department of Data Science and Engineering



School of Computer Science and Engineering,

MANIPAL UNIVERSITY JAIPUR
RAJASTHAN, INDIA

October 2026

भारतीय प्रौद्योगिकी संस्थान हैदराबाद
कंडी - ५०२ २८५, संगारेड्डी, तेलंगाना, भारत
फोन : (०४०) २३०१ ६०३३; फैक्स : (०४०) ६००३ / ३२

Indian Institute of Technology Hyderabad
Kandi – 502 285, Sangareddy, Telangana, INDIA
Phone: (040) 2301 6033; Fax: (040) 2301 6003 /32

TO WHOM IT MAY CONCERN

This is to certify that **Ayush Gupta**, a student of B.Tech in Computer Science at Manipal University, successfully completed his **Summer Internship** under the supervision of **Dr. Prasad S. Onkar** at the INDReA Lab, Department of Design, Indian Institute of Technology Hyderabad, from 1 June 2026 to 31 July 2026.

During the internship, he worked on the development of a meditation kiosk inspired from *Diya* – a traditional Indian lamp. Throughout the internship, he demonstrated strong research aptitude, consistency, and the ability to work both independently and collaboratively as part of a team. His conduct and performance were **excellent** throughout the internship.

We wish him success in his future academic and professional endeavours.

07/10/2026

Prasad S. Onkar
Associate Professor,
Department of Design
Indian Institute of Technology Hyderabad
Email: psonkar@des.iith.ac.in

ACKNOWLEDGMENTS

I would like to express my heartfelt gratitude to **Dr. Prasad S. Onkar**, Associate Professor, Department of Design, Indian Institute of Technology Hyderabad, for giving me the opportunity to work at the INDReA Lab and for his supervision and encouragement throughout my internship from 1 June 2026 to 31 July 2026.

I am deeply thankful to **Mr. Eshwar Teja**, under whose day-to-day guidance this project was carried out. His design discussions, his testing of every build on the real kiosk and his patience while the system was taking shape were instrumental in the successful completion of my work. I also thank **Kalyan Sir** for his support and valuable feedback, and the members of the hardware and computer-vision team for their cooperation in defining clear interfaces between our parts of the system.

I sincerely thank my faculty guide, **[[Faculty guide name]]**, and the Department of Data Science and Engineering, School of Computer Science and Engineering, **Manipal University Jaipur**, for their guidance and for providing me with the opportunity to undertake this industrial training.

Finally, I am grateful to my family and friends for their constant encouragement and support throughout the internship.

ABSTRACT

Diya is an unattended meditation kiosk developed at the INDReA Lab, Department of Design, IIT Hyderabad, and named after the traditional Indian oil lamp. A visitor walks up to the kiosk, is identified, is aligned to a set of cameras by a motorised rig, sits through a guided meditation while thermal, depth, gaze and posture data are captured, and receives a personalised report on screen. The machine then resets itself for the next person without any operator.

During this eight-week internship I designed and built the software side of the kiosk: a full-screen desktop application in C# on .NET 8 with the Avalonia UI framework for Ubuntu (GNOME on Wayland); a Node.js/Express and PostgreSQL backend hosted on Render; phone-facing web pages for registration, roster administration and login; an integration layer that runs the hardware team's calibration and meditation pipeline and displays the PDF report it produces; and packaging that ships the kiosk as a self-contained Debian package that starts on boot and restarts after a crash.

The key design decision was to **invert the usual QR-code flow**. The kiosk had no scanner, so the kiosk displays a short-lived session QR code and the visitor's own phone scans it through a private, per-person login link. This removed a hardware dependency, needed no app installation and made the scan a proof of who arrived. A second decision was to integrate with the hardware team through a process and a file on disk rather than an API, which let both teams work independently.

The system went through seven released versions (v1.0 to v1.6) and was demonstrated end to end with a real phone scanning the kiosk. The report also discusses the problems faced, in particular the window restrictions of Wayland, and the remaining work: a consent screen, an inactivity reset, continuous integration and replacing the placeholder calibration scripts with the hardware team's real stages.

LIST OF FIGURES

Figure No	Figure Title	Page No
1)	Overall system architecture of the Diya kiosk	10
2)	Hardware block diagram of the kiosk	11
3)	Visitor flow of the kiosk as a state machine	11
4)	Sequence of interactions between admin, phone, kiosk, backend and meditation-app	12
5)	Database schema (entity–relationship view)	12
6)	Retry logic of the pipeline script run1.sh	13
7)	Build and deployment of the kiosk and the backend	13
8)	Web pages of the system (sample data)	15
9)	Front view of the Diya meditation kiosk	17
10)	Side view showing the seat and sensor arm	18
11)	Camera assembly	18
12)	Motorised chest and eye axes	19
13)	Arduino controller and wiring	19
14)	LD2410 radar sensor and mounting	20
15)	Kiosk idle: session created and QR code displayed	20
16)	Kiosk after the phone has claimed the session	21
17)	Pipeline running with the live status line	21
18)	Report displayed with the thank-you message and Return	22
19)	Live demonstration: the visitor’s phone	22
20)	Live demonstration: the kiosk after the claim (email blurred)	23
21)	Timeline of the main versions	25
22)	Gantt chart of the internship (1 June – 31 July 2026)	31
23)	Wireframe WF-01: kiosk idle, waiting to be scanned	32
24)	Wireframe WF-02: visitor identified, session starting	33
25)	Wireframe WF-03: report overlay	33
26)	Wireframe WF-04: the visitor’s three taps on the phone	34

LIST OF TABLES

Table No	Table Title	Page No
1)	Division of responsibilities	1
2)	Comparison of visitor identification approaches	5
3)	Comparison of desktop UI frameworks	7
4)	Functional requirements	8
5)	Non-functional requirements	8
6)	REST endpoints of the backend	14
7)	Configuration through environment variables	16
8)	Hardware components of the kiosk	17
9)	Version history	25
10)	Summary of test cases	26
11)	Main challenges and how they were addressed	27
12)	Weekly work log	31

Contents

		Page No
Acknowledgement		i
Abstract		ii
List Of Figures		iii
List Of Tables		iv
List of Abbreviations		vi
Chapter 1	**INTRODUCTION**	1
1.1	Introduction to work done/ Motivation (Overview, Applications & Advantages)	1
1.2	Project Statement / Objectives of the Project	3
1.3	Organization of Report	4
Chapter 2	**BACKGROUND MATERIAL**	5
2.1	Conceptual Overview (Concepts/Theory used)	5
2.2	Technologies Involved	7
Chapter 3	**METHODOLOGY**	8
3.1	Detailed methodology that will be adopted	8
3.2	Circuit Layouts / block diagrams	10
Chapter 4	**IMPLEMENTATION**	14
4.1	Modules	14
4.2	Prototype	17
Chapter 5	**RESULTS AND ANALYSIS**	24
5.1	Results	24
5.2	Testing and Validation	26
5.3	Challenges Faced and Learnings	27
Chapter 6	**CONCLUSIONS & FUTURE SCOPE**	28
6.1	Conclusions	28
6.2	Future Scope of Work (at least 3 points)	29
REFERENCES		**30**
ANNEXURES		**31**

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CI	Continuous Integration
CV	Computer Vision
.deb	Debian software package
GNOME	GNU Network Object Model Environment (Linux desktop)
HTTPS	Hypertext Transfer Protocol Secure
IITH	Indian Institute of Technology Hyderabad
INDReA	Interaction Design Research Lab (IITH Department of Design)
JSON	JavaScript Object Notation
LTS	Long-Term Support
PDF	Portable Document Format
PR	Pull Request
QR	Quick Response code
REST	Representational State Transfer
SQL	Structured Query Language
UI / UX	User Interface / User Experience
V4L2	Video4Linux version 2
XLSX	Office Open XML spreadsheet (Excel)

INTRODUCTION

1.1) Introduction to work done/ Motivation (*Overview, Applications & Advantages*)

The internship was carried out at the **INDReA Lab, Department of Design, Indian Institute of Technology Hyderabad**, under the supervision of Dr. Prasad S. Onkar, from 1 June 2026 to 31 July 2026. The lab was developing *Diya*, a meditation kiosk inspired by the traditional Indian lamp, and I joined the project to build its software.

Overview: The Diya kiosk is an installation meant for a public, museum-like space. A visitor walks up, is identified, and is guided through a short meditation. During the session several cameras observe the person (a webcam for face and gaze, a depth camera, a posture camera and a thermal camera), and a motorised rig first aligns these sensors to the visitor's chest and eyes. At the end, a report of the session is shown on screen and the kiosk returns to its welcome screen, ready for the next visitor, with no attendant.

My work covered the complete software path that a visitor experiences: the full-screen kiosk application, the visitor identification system (backend, database and phone web pages), the layer that launches the hardware team's pipeline and shows its report, and the packaging and deployment of the kiosk. The camera and motor control scripts and the meditation application were built by a collaborating hardware and computer-vision team (Table 1).

Table 1: Division of responsibilities

Party	**Owns**	**Interface to the kiosk app**
Me (this internship)	Kiosk desktop app, backend API and database, web pages for registration, admin and login, pipeline launcher, report display, packaging	—
Hardware / CV team	Camera calibration and motor scripts (HOME, SHOOT, CHEST, EYE) and the `meditation-app` that runs the session and writes the PDF report	Script `run1.sh` and the folder `/opt/meditation-app/data`
Lab / supervisors	Concept, experience design, hardware rig, testing on the real kiosk	Requirements and review

Motivation: An unattended kiosk has to do on its own what an attendant would normally do: greet the visitor, find out who they are, start the right process, recover when something fails and get ready for the next person. For Diya this had to work on a Linux machine with several cameras attached, alongside software written by another team, and without a QR scanner or keyboard for the visitor.

Applications: The same software pattern applies to museum and exhibition installations, wellness and meditation booths, self-service health check-up kiosks, and any setting where a person must be identified at a shared screen and a local hardware process must then run for them.

Advantages: Identification needs nothing but the visitor's own phone; the kiosk recovers

from crashes by itself and installs on a fresh machine with one command; and the clean boundary with the hardware software allows the two teams to change their internals independently.

1.2) Project Statement / Objectives of the Project

Project statement: *To design and build reliable, self-recovering kiosk software that identifies a visitor without dedicated hardware, runs the hardware team's calibration and meditation pipeline, shows the resulting report and resets automatically for the next visitor, while remaining easy to install and update on the kiosk machine.*

The objectives of the project were:

1. **Kiosk application:** a full-screen application for Ubuntu (GNOME on Wayland) that starts on boot and restarts itself after a crash.
2. **Visitor identification:** a method that needs no scanner or keyboard at the kiosk.
3. **Backend and web pages:** visitor registration, roster upload by an administrator and phone-based login.
4. **Pipeline integration:** run the hardware team's calibration scripts and meditation application without depending on their internal code.
5. **Report display:** show the generated PDF report inside the kiosk and reset for the next visitor.
6. **Packaging and documentation:** install on a fresh machine with a single command, with documented installation, operation and troubleshooting.

Building the camera and motor control logic and the meditation content was outside the scope of this internship; these were provided by the hardware team and are described only where the kiosk software had to be designed around them.

1.3) Organization of Report

This report has been structured to give a systematic account of the work done during the internship at the INDReA Lab, IIT Hyderabad. The chapters are organised as follows:

1. **Introduction:** the project, its motivation, the problem statement and the objectives.
2. **Background Material:** the concepts behind an unattended kiosk and the technologies used, with the reasons for choosing them.
3. **Methodology:** the step-by-step approach followed, and the block diagrams of the system, the hardware, the visitor flow, the database and the pipeline.
4. **Implementation:** the software modules and the prototype, including the kiosk apparatus and the screens of the final application.
5. **Results and Analysis:** the outcomes, the testing carried out, the evolution of the design, and the challenges faced and lessons learned.
6. **Conclusions & Future Scope:** a summary of the achievements and realistic extensions of the work.
7. **References and Annexures:** sources used, the weekly work log, code snippets, wireframes and setup commands.

BACKGROUND MATERIAL

2.1) Conceptual Overview (*Concepts/Theory Used*)

Several concepts from human–computer interaction, distributed systems and operating systems were applied in this project. They are summarised below.

1. Unattended kiosk systems

Kiosk software differs from an ordinary desktop application mainly in what happens when nobody is watching. It must start without a login prompt, occupy the whole screen, survive crashes and power cuts, keep a visitor from reaching the desktop and return to a known state after every visitor. Every failure must either be recovered automatically or be shown clearly enough for staff to act on it.

2. Visitor identification and QR codes

A QR code is a two-dimensional barcode that can carry a short text such as a URL, and every modern phone camera can read it. Several ways of identifying a visitor were compared (Table 2). Because the kiosk had no scanner and its cameras were reserved for the meditation pipeline, the chosen approach was for the **kiosk to show** a QR code and the **phone to scan** it.

Table 2: Comparison of visitor identification approaches

Approach	**Hardware at kiosk**	**Visitor effort**	**Remarks**
Printed pass + USB scanner	QR scanner	Carry and present a pass	Extra hardware and printing; explored as an offline variant
Typing details at the kiosk	Keyboard	High	Kept only as a staff/testing fallback
Phone shows QR to kiosk camera	Camera facing visitor	Medium	Kiosk cameras are needed by the CV pipeline
Kiosk shows QR, phone scans	None	Low: open link, tap, scan	**Chosen**
Face recognition	Camera	None	Future option; needs enrolment and privacy care

3. Client–server communication and polling

The kiosk and the phone never talk directly. Both talk to a REST API over HTTPS. The kiosk creates a *session* and then **polls** the server every two seconds to learn whether a phone has claimed it. Polling is simpler than a push channel, works through any network and is cheap at this scale.

4. Display servers and the Wayland security model

Ubuntu’s GNOME desktop uses **Wayland**. Unlike the older X11 system, the compositor decides where windows go, and an application may not move, raise or hide another application’s windows, or even raise itself. This improves security but limits what kiosk software can do, and several design decisions followed from it.

5. Process-based integration

Instead of an API between the kiosk and the hardware software, the integration uses two operating-system primitives: a **child process** (the kiosk runs a shell script and waits for it to exit) and a **file on disk** (the meditation application writes a PDF that the kiosk reads). This loose coupling lets each team change its internals without breaking the other.

6. Packaging, services and fault recovery

A self-contained Debian package bundles the application with its runtime, so nothing has to be installed first. A **systemd user service** starts the kiosk with the desktop session and restarts it automatically if it exits, which provides crash recovery without custom code.

7. Privacy and security

The roster contains names, photographs and Aadhaar numbers, so the design follows data minimisation: links use long random tokens that cannot be guessed, public endpoints return only what is needed to greet a person, and the administrator’s upload is protected by a secret key.

2.2) Technologies Involved

The following tools and technologies were used to build, deploy and test the system:

1. Kiosk desktop application

- **C# and .NET 8 (LTS):** garbage-collected memory removes a whole class of crashes for a machine that runs all day; publishing produces one self-contained binary.
- **Avalonia 11.2.3:** a cross-platform .NET UI framework that officially supports Linux and draws its own controls with Skia, so the UI looks identical on the laptop and the kiosk.
- **QRCoder 1.6.0:** generates the session QR code without System.Drawing.
- **PDFtoImage 4.1.1 (PDFium + SkiaSharp):** renders the report pages inside the application.

2. Backend and database

- **Node.js 18+ and Express 4:** a small REST API that also serves the web pages.
- **PostgreSQL with the pg driver:** stores visitors, sessions and the roster.
- **Render:** hosts the web service and database from a blueprint file, with HTTPS.

3. Web pages

- **HTML and JavaScript:** registration, admin and login pages.
- **SheetJS:** parses the uploaded Excel roster inside the browser.
- **html5-qrcode:** turns the phone camera into a QR scanner.

4. Pipeline, packaging and tools

- **Bash and Python 3.10:** the pipeline script and the hardware team's calibration scripts.
- **dotnet publish, dpkg-deb and systemd:** the Debian package, auto-start and restart.
- **Git and GitHub:** one branch and pull request per change.
- **Docker, Xvfb and noVNC:** preview of the UI in a browser during development.

Table 3 compares the UI frameworks that were considered for the kiosk.

Table 3: Comparison of desktop UI frameworks

Option	Linux support	Native access	Verdict
C# / .NET 8 + Avalonia	Official (X11 and Wayland)	Full	**Chosen**
.NET MAUI	No Linux target	Full	Rejected
Electron	Good	Needs a native helper	Heavier; two processes
Web app in a browser	Good	None (sandbox)	Cannot launch the pipeline
C++ with Qt	Good	Full	No benefit for this UI; manual memory

METHODOLOGY

3.1) Detailed methodology that will be adopted

1. Requirement Analysis

- **Discussions with the lab:** understood the visitor experience, the hardware rig and what the hardware team would provide.
- **Constraints identified:** no QR scanner or keyboard at the kiosk; cameras reserved for the pipeline; Ubuntu with GNOME on Wayland; unattended operation.
- **Requirements written down:** functional requirements (Table 4) and non-functional requirements (Table 5).

Table 4: Functional requirements

ID	Requirement
FR1	On start-up the kiosk opens fullscreen and shows a welcome screen with a QR code.
FR2	Each QR code carries a fresh, hard-to-guess session token created by the backend.
FR3	An administrator uploads an Excel roster and gets one private login link per person.
FR4	A visitor opens their link on a phone, scans the kiosk QR and is identified.
FR5	Visitors not on the roster can register themselves on the phone.
FR6	The kiosk shows the visitor’s name and photo within a few seconds of the scan.
FR7	The kiosk then runs the calibration and meditation pipeline automatically, with a live status.
FR8	When the pipeline ends, the kiosk shows the new report and a thank-you message.
FR9	A Return button resets the kiosk and creates a new session.
FR10	A manual name entry lets staff start a session without a phone or server.

Table 5: Non-functional requirements

Category	Requirement
Reliability	Auto-start on boot; restart within seconds after a crash; no state between visitors.
Recoverability	Network failures show a clear message and a retry action.
Portability	Installs on any recent Ubuntu (amd64 or arm64) with nothing pre-installed.
Privacy	Aadhaar never returned by public endpoints; unguessable tokens.
Usability	At most three taps on the phone; nothing to type at the kiosk.
Maintainability	Editable pipeline script; paths overridable by environment variables.

2. System Design

- Designed the architecture, the visitor flow as a state machine, the database schema and the REST API (Section 3.2).
- Designed the screens as wireframes before coding them (Annexure C).
- Agreed the integration contract with the hardware team: one script to run and one

folder to read the report from.

3. Iterative Development

- Built the system in small increments; every change on its own Git branch and merged through a pull request (28 pull requests during the internship).
- Released a new Debian package whenever the kiosk needed an update, and updated the setup guide with every merge.
- Explored alternatives on separate branches (an offline people-list mode and an in-app meditation flow) before choosing the final direction.

4. Integration with the Hardware Pipeline

- Wrapped the hardware team's stages in an editable script with retry rules.
- Displayed only reports written after the session started, so a previous visitor's report can never be shown.

5. Testing and Validation

- Automated end-to-end tests of the backend session flow and the kiosk's HTTP client.
- Headless rendering of every screen; a no-camera test kit that replaces the pipeline with a mock.
- Installation and live runs on the real kiosk, including a recorded demonstration with a phone.

6. Deployment and Documentation

- Packaged the kiosk as a self-contained .deb with a systemd service and kiosk setup script.
- Deployed the backend on Render from a blueprint file.
- Wrote the setup guide, a handoff document and a codebase overview for the lab.

3.2) Circuit Layouts / block diagrams

1. Overall System Architecture

Block diagram elements: the administrator's PC and the visitor's phone (browsers); the backend API and PostgreSQL database on Render; and, on the kiosk machine, the Diya application, the pipeline script and the hardware team's stack (Figure 1).

Flow: Admin PC → Backend (roster) · Kiosk → Backend (create session, poll) · Phone camera reads kiosk QR → Backend (claim) · Kiosk → run1.sh → hardware stack → report PDF → Kiosk.

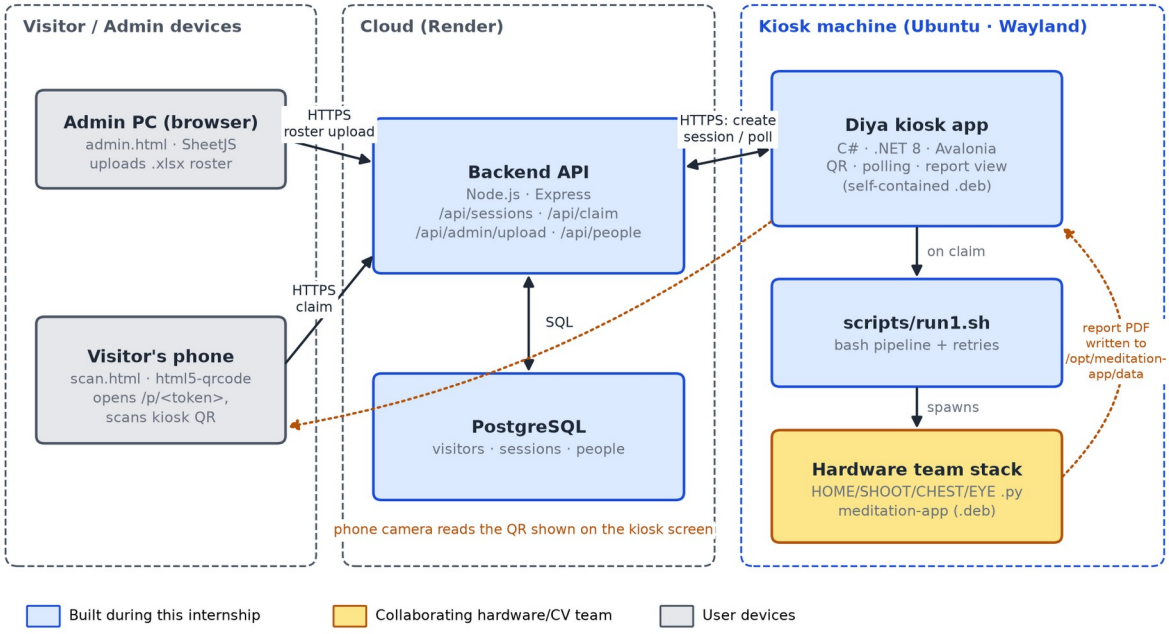


Figure 1: Overall system architecture of the Diya kiosk

The kiosk and the hardware stack never call each other over an API; the only contract is a script to run and a folder to watch.

2. Hardware Block Diagram

Block diagram elements: four USB cameras (Logitech C920, Intel RealSense D435, Arducam 8MP and a thermal camera), the display, an Arduino controller driving motors M1 (chest axis) and M2 (eye axis), the limit switches and proximity sensors, and an LD2410 radar that detects a seated person (Figure 2).

Flow: Cameras → Kiosk PC (V4L2) · Kiosk PC ⇄ Arduino (USB serial, 115200 baud) → Motors · Safety inputs → Arduino · Radar → Kiosk PC (USB serial, 256000 baud) · Kiosk PC → Display (HDMI).

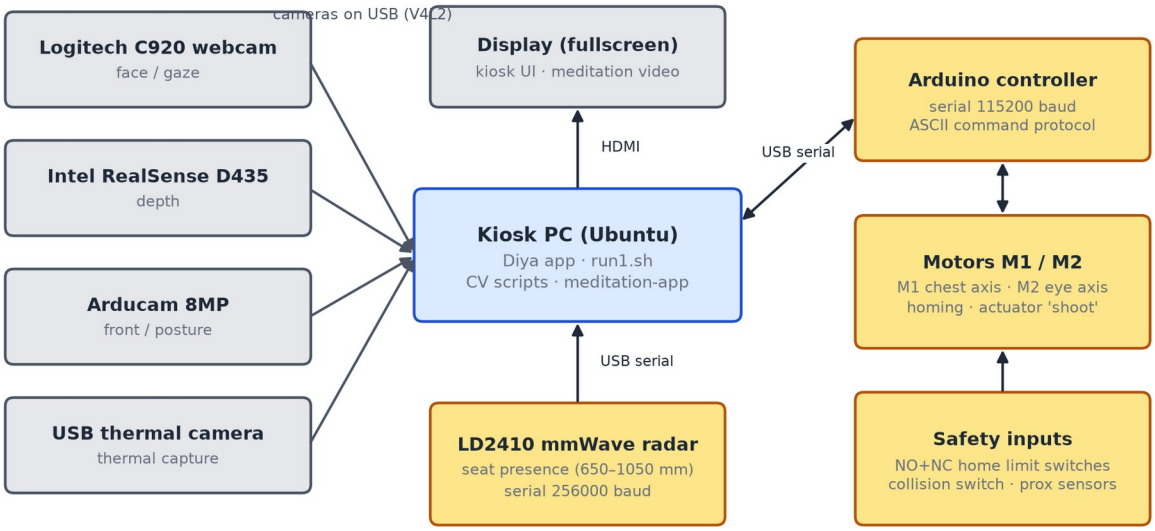


Figure 2: Hardware block diagram of the kiosk

3. Visitor Flow

Flow: Idle (QR on screen) → Claimed (name and photo) → Session (pipeline running) → Report (PDF and thank-you) → Return → Idle. A manual name entry skips straight to Claimed (Figure 3).

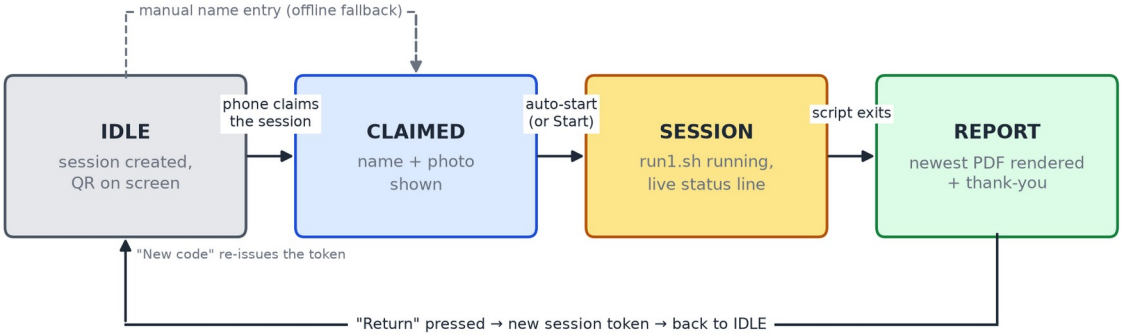


Figure 3: Visitor flow of the kiosk as a state machine

4. Interaction Sequence

Figure 4 shows every message for one visitor. Solid arrows are HTTP requests, dashed arrows are responses, and dotted arrows do not use the API at all: the link shared with the visitor, the phone camera reading the screen, and the local pipeline with its PDF output.

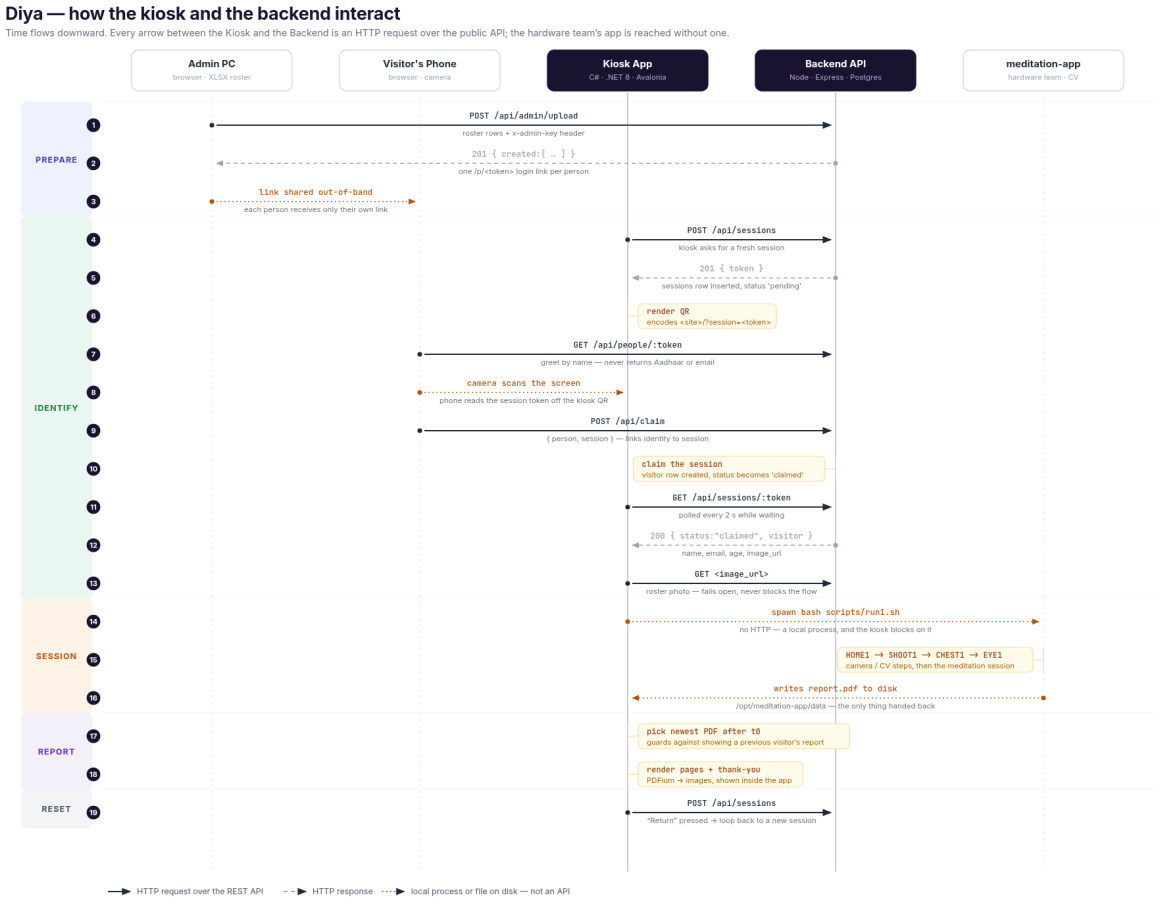


Figure 4: Sequence of interactions between admin, phone, kiosk, backend and meditation-app

5. Database Design

people holds the uploaded roster; each row has an unguessable token that forms the private link `/p/<token>`. **sessions** holds one row per QR code shown on the kiosk, with status pending or claimed. **visitors** holds the identity linked to a session; on a claim the roster details are copied into a visitor row (Figure 5).

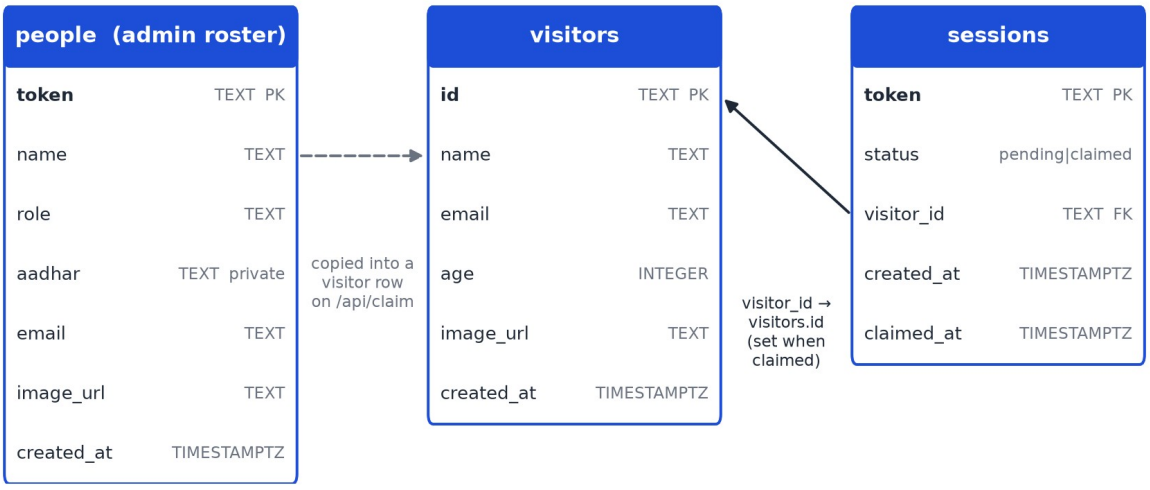


Figure 5: Database schema (entity–relationship view)

6. Pipeline Flowchart

Flow: HOME1 (home the axes) → SHOOT1 (deploy actuators) → CHEST1 (align chest camera) → EYE1 (align eye camera) → meditation-app (session and PDF report). Homing and deployment retry themselves; a failure in chest or eye alignment restarts the whole sequence from homing (Figure 6).

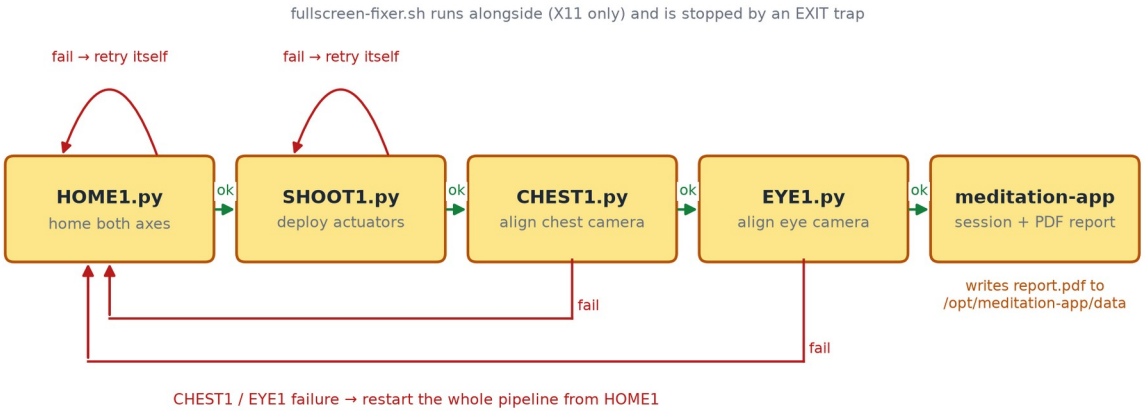


Figure 6: Retry logic of the pipeline script run1.sh

7. Build and Deployment

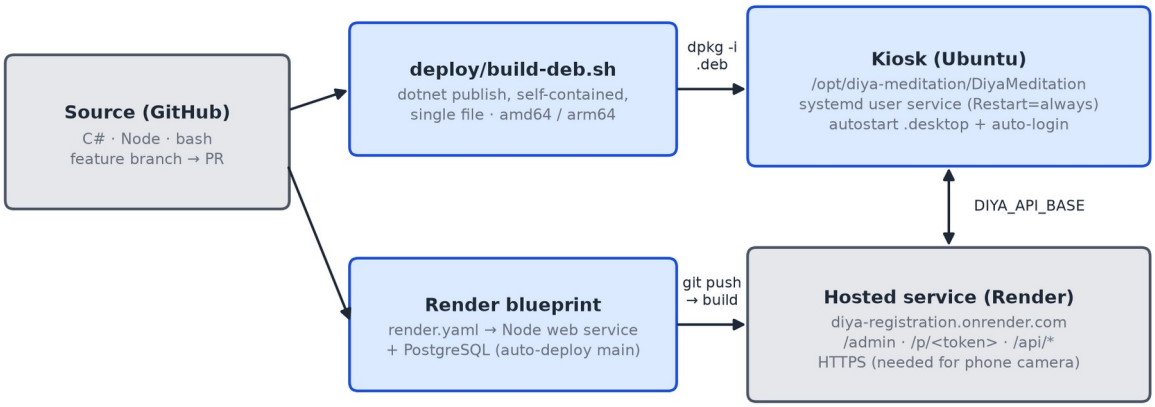


Figure 7: Build and deployment of the kiosk and the backend

IMPLEMENTATION

4.1) Modules

The implementation was divided into the following modules:

1. Kiosk Shell Module (fullscreen on Wayland)

- **Resizable window:** marking the window non-resizable sets fixed size hints, and the compositor then refuses to make it fullscreen.
- **Fullscreen applied in code and re-asserted:** applied when the window opens and re-applied six times at 250 ms intervals, because the compositor first maps the window in its normal state.
- **Portable build:** invariant globalisation removes the libicu dependency, and a self-contained single-file publish bundles the .NET runtime.
- **Delayed auto-start:** launching before the Wayland session is ready fails silently, so start-up waits a few seconds.

2. Home Screen, Session QR and Polling Module

- Requests a new session; because the free server sleeps when idle, the status reads "Waking up the server…" and the HTTP client allows 60 seconds.
- Encodes `<site>/?session=<token>` as a QR image and polls `GET /api/sessions/:token` every 2 seconds.
- On a claim, shows the visitor's name, email, age and roster photo; the photo fails open and never blocks the session.
- Offers **New code** to retry after a network error, and a manual name entry for staff.

3. Backend API Module

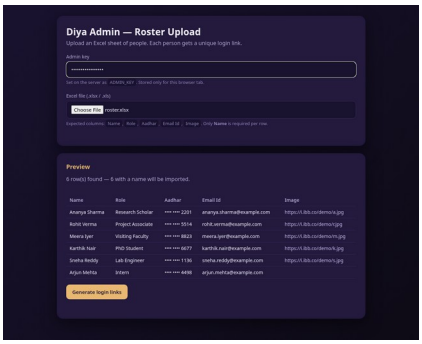
- **Unguessable tokens:** session tokens of 9 random bytes and personal link tokens of 16 random bytes.
- **Claim only once:** a session is linked to a visitor only while its status is still pending.
- **Minimal disclosure:** the greeting endpoint returns only name and role.
- **Admin protection:** roster upload requires the ADMIN_KEY secret; without it the feature is off.

Table 6: REST endpoints of the backend

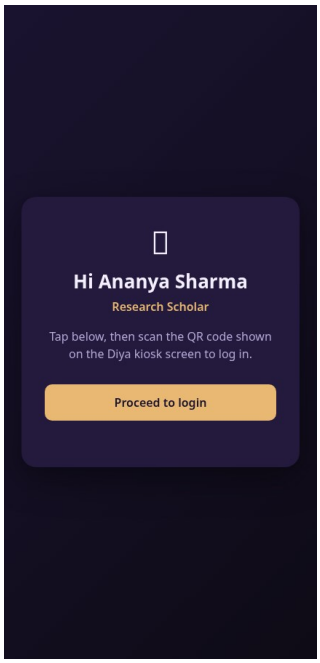
Method	Endpoint	Used by	Purpose
GET	/api/health	Operators	Liveness check
POST	/api/sessions	Kiosk	Create a session; returns a token
GET	/api/sessions/:token	Kiosk	Poll: pending, or claimed with the visitor
POST	/api/visitors	Phone	Self-registration
POST	/api/admin/upload	Admin page	Create people from the roster (needs key)
GET	/api/people/:token	Phone	Greeting: name and role only
POST	/api/claim	Phone	Link the person to the session

4. Web Pages Module

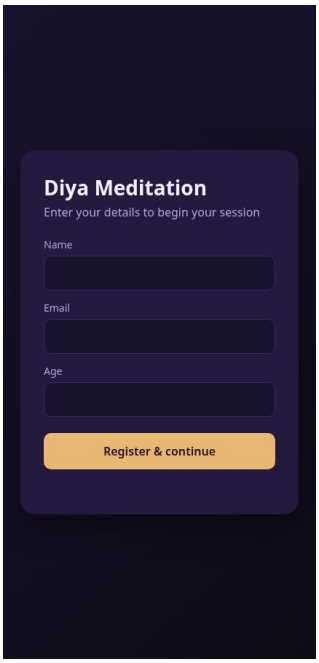
- **admin.html:** parses the Excel roster in the browser, matches column headers loosely, previews rows with Aadhaar masked, and returns one personal link per person with Copy all and CSV download.
- **scan.html (/p/<token>):** greets the person, opens the phone camera, reads the kiosk QR and claims the session.
- **index.html:** self-registration for visitors who are not on a roster.



(a) Admin roster upload



(b) Personal login link



(c) Self-registration

Figure 8: Web pages of the system (sample data)

5. Pipeline Integration and Report Display Module

- Runs `bash scripts/run1.sh` without a terminal window and shows each line it prints as the live status; there is deliberately no timeout because the meditation takes several minutes.
- Picks the newest PDF in `/opt/meditation-app/data` written **after the session started**, so a previous visitor's report is never shown.
- Renders the pages as images inside the app, so no external viewer can take focus, and adds a thank-you line and a single Return button.

6. Packaging and Deployment Module

- `build-deb.sh` builds a self-contained .deb for amd64 or arm64 with the scripts, desktop entry and systemd unit.
- The systemd user service restarts the app 2 seconds after any exit; `setup-kiosk.sh` enables auto-login and disables escape shortcuts.
- The backend is deployed from `render.yaml`, which creates the web service and database.

Table 7: Configuration through environment variables

Variable	Where	Meaning
DIYA_API_BASE	Kiosk	Backend URL
DIYA_PIPELINE_SCRIPT	Kiosk	Script to run on login
DIYA_REPORT_DIR	Kiosk	Folder to read the report from
DATABASE_URL	Server	PostgreSQL connection string
ADMIN_KEY	Server	Secret that enables and protects roster upload

4.2) Prototype

1. Kiosk Apparatus

The prototype is a physical installation: a seat, a display, and a sensor assembly carrying the cameras, moved by two motorised axes and guarded by limit switches, proximity sensors and a radar. The apparatus and its control scripts were built by the hardware team; Table 8 lists the components.

Table 8: Hardware components of the kiosk

Component	Role	Interface
Kiosk PC (Ubuntu)	Runs the kiosk app, CV scripts and meditation-app	—
Display	Kiosk UI, meditation video, report	HDMI
Logitech C920 webcam	Face and gaze	USB (V4L2)
Intel RealSense D435	Depth	USB
Arducam 8MP	Front view and posture	USB (V4L2)
USB thermal camera	Thermal capture	USB (V4L2)
Arduino controller	Drives motors; reads switches and sensors	USB serial, 115200 baud
Motors M1 and M2	Chest axis and eye axis; homing and actuator	Via Arduino
Limit switches, proximity sensors	Home position, collision detection	Via Arduino
LD2410 mmWave radar	Seated person, 650–1050 mm	USB serial, 256000 baud



Figure 9: Front view of the Diya meditation kiosk

PLACE PHOTOGRAPH HERE

Insert photograph: seating position relative to the cameras.

Figure 10: Side view showing the seat and sensor arm

PLACE PHOTOGRAPH HERE

Insert photograph: close-up of the camera mounts.

Figure 11: Camera assembly

PLACE PHOTOGRAPH HERE

Insert photograph: motors, axes and limit switches.

Figure 12: Motorised chest and eye axes

PLACE PHOTOGRAPH HERE

Insert photograph: controller board and wiring.

Figure 13: Arduino controller and wiring

PLACE PHOTOGRAPH HERE

Insert photograph: radar module and its position.

Figure 14: LD2410 radar sensor and mounting

2. Screens of the Final Application

Figures 15 to 18 are genuine frames of the kiosk application rendered with the same layout, fonts and QR encoder as on the kiosk. The visitor is fictional and the report is a representative sample, since the real report comes from the hardware team’s software.

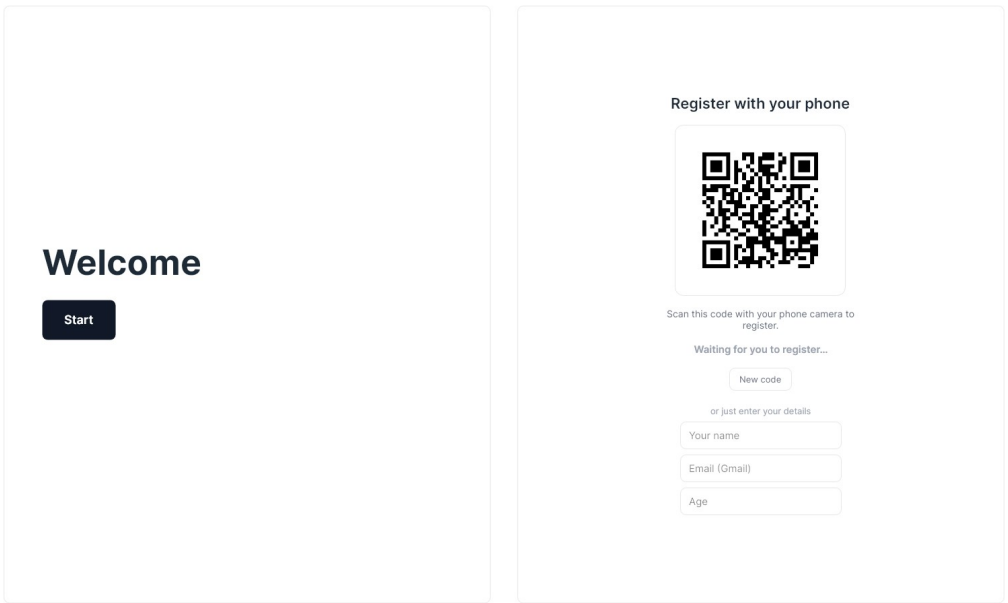


Figure 15: Kiosk idle: session created and QR code displayed

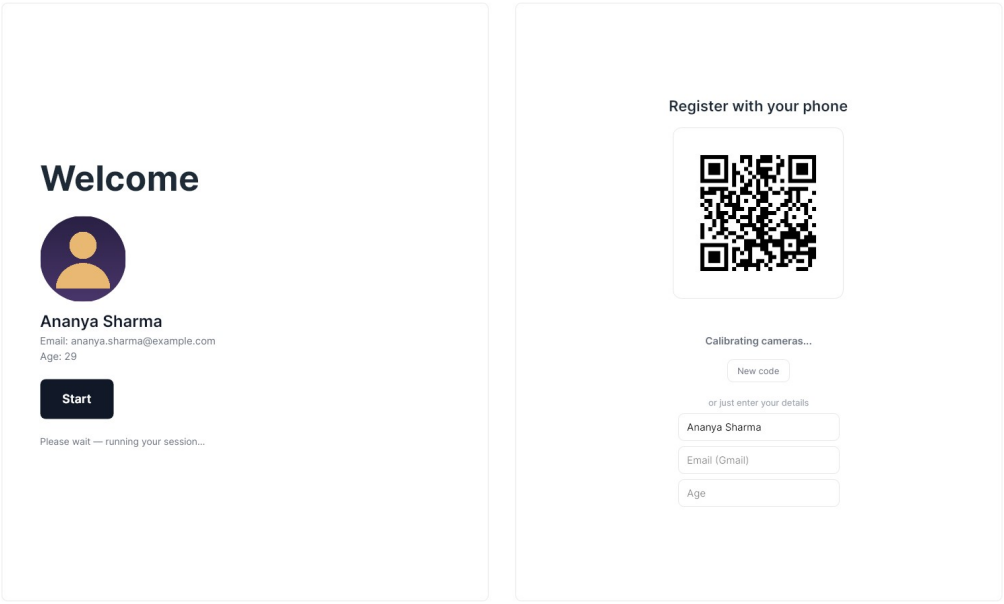


Figure 16: Kiosk after the phone has claimed the session

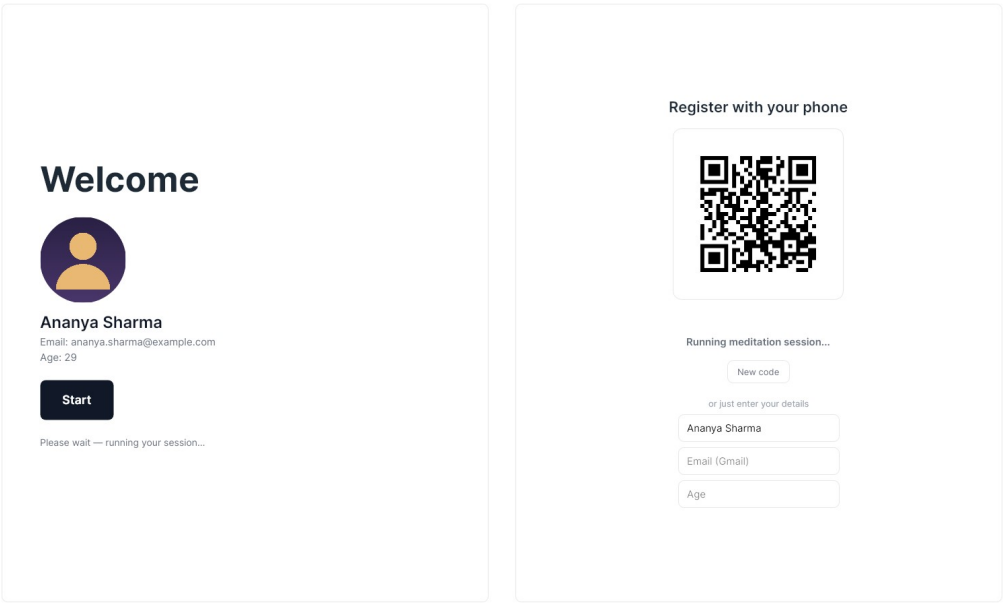


Figure 17: Pipeline running with the live status line

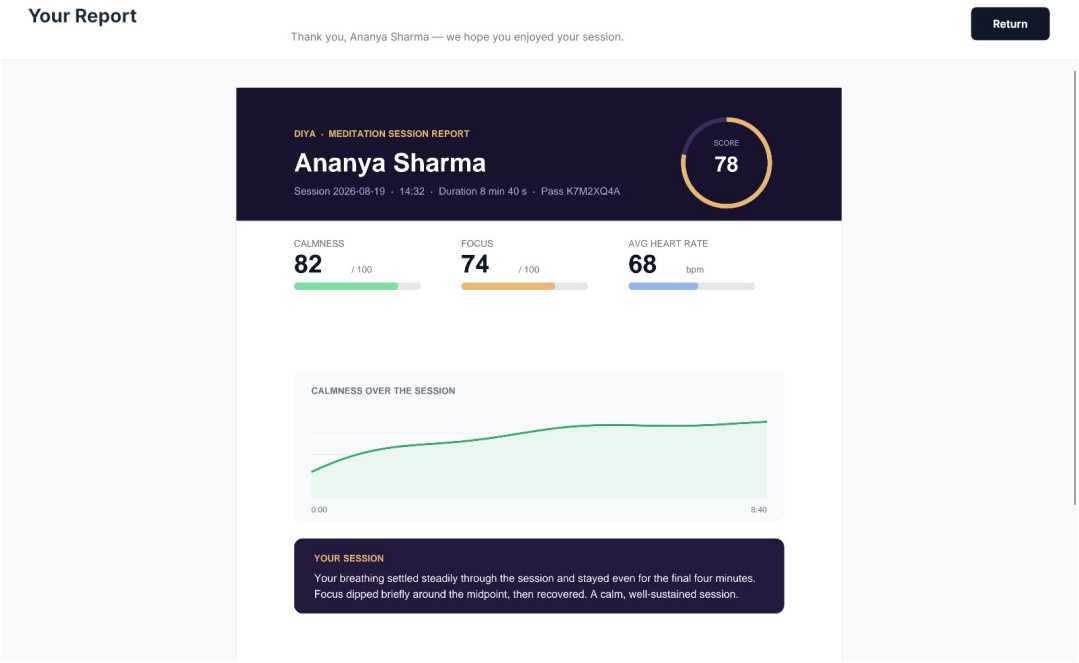
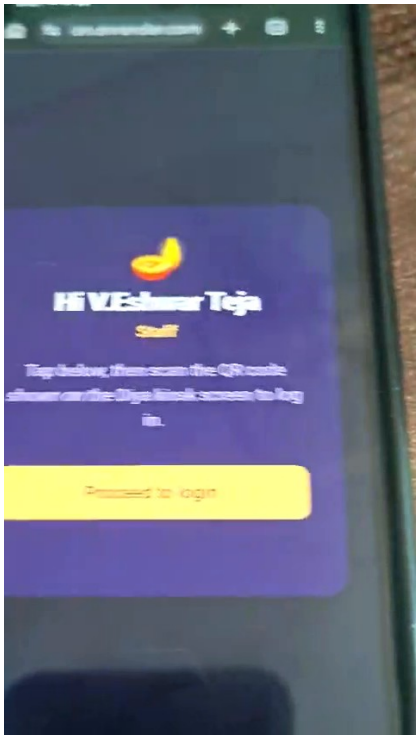


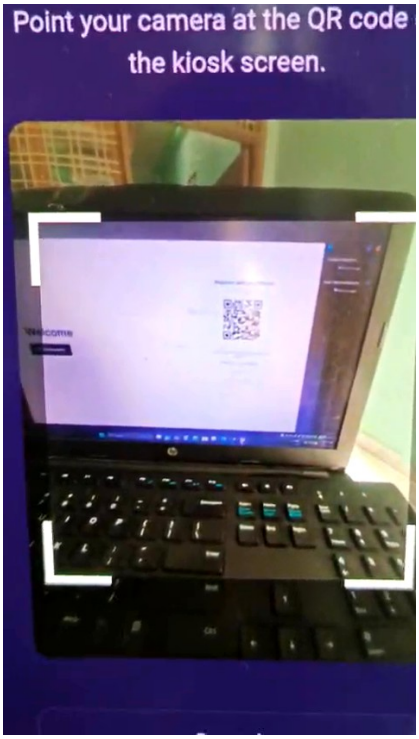
Figure 18: Report displayed with the thank-you message and Return

3. Live Demonstration

Figures 19 and 20 are frames from a recorded run with the deployed backend: the personal link greets the visitor, the phone scans the kiosk QR, and the kiosk advances by itself. The recording used an earlier build, so some labels differ slightly from the final version.



(a) Personal link greets the visitor



(b) Phone scanning the kiosk QR

Figure 19: Live demonstration: the visitor’s phone

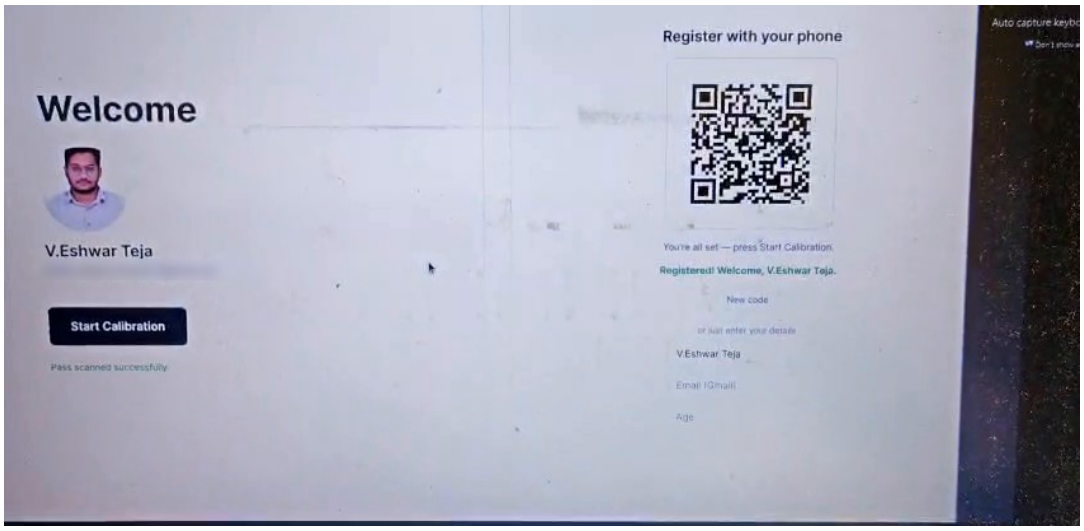


Figure 20: Live demonstration: the kiosk after the claim (email blurred)

RESULTS AND ANALYSIS

5.1) Results

1. Visitor Identification Without Extra Hardware

Results:

- Visitors are identified with only their own phone: open the link, tap, scan.
- Each scan proves who arrived, because the link is private and unguessable.
- The kiosk shows the visitor's details within one polling interval (2 s) of the claim.

Analysis: The missing scanner turned out to simplify the design. No app, no typing at the kiosk and no extra hardware were needed, and the kiosk cameras stayed free for the meditation pipeline.

2. Reliable Unattended Operation

Results:

- The kiosk opens fullscreen on GNOME/Wayland and installs from one package on amd64 and arm64.
- systemd restarts the application after any crash; network errors show a clear retry option.

Analysis: Building on the platform's own mechanisms (systemd, self-contained publishing) gave reliability with very little custom code, which suits a machine that runs without supervision.

3. Integration with the Hardware Team

Results:

- The complete journey (identify, claim, pipeline, report, reset) works end to end.
- A previous visitor's report is never shown, and no terminal or viewer window appears from the kiosk.

Analysis: A contract of one script and one folder was enough, and it let both teams develop and test independently.

4. Evolution of the Design

The design changed most in how visitors are identified (Figure 21, Table 9). The first version put the visitor's data inside the QR; this was readable by anyone and could not be revoked, so version 1.2 stored the data online and kept only an ID in the code. Version 1.3 reversed the flow so the kiosk shows the QR, and the admin roster added in July made each scan prove identity. In late July the kiosk was made to minimise itself during the session, but Wayland does not let an application restore itself, so the change was rolled back to the last stable baseline and the useful parts were re-implemented in version 1.6.0.

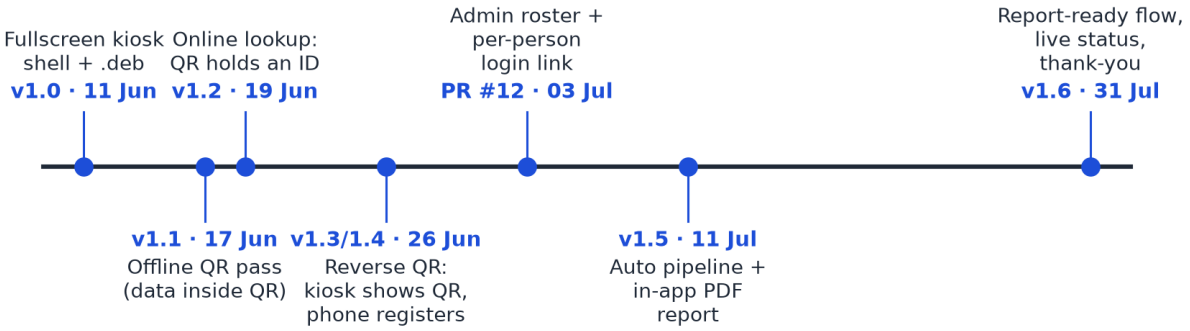


Figure 21: Timeline of the main versions

Table 9: Version history

Version	Date	Change
1.0.x	11–13 Jun	Fullscreen kiosk shell, Wayland fixes, .deb for amd64 and arm64
1.1.0	17 Jun	Offline QR pass: visitor details inside the QR
1.2.0	19 Jun	Online lookup: QR carries only an ID
1.3.0	26 Jun	Reverse flow: kiosk shows the QR, phone registers
1.4.0	26 Jun – 9 Jul	Calibration launch, admin roster, phone login, visitor photo
1.5.0	11 Jul	Login runs the pipeline and shows the PDF report
1.6.0	31 Jul	Report-ready flow, live status, thank-you, no-camera test kit

5.2) Testing and Validation

Table 10: Summary of test cases

#	Scenario	Expected result	Outcome
T1	Create session, register, poll	Poll returns claimed	Passed (automated)
T2	Kiosk client: claim and unknown token	Claimed; unknown is NotFound	Passed (automated)
T3	Render every screen headlessly	Screens build	Passed
T4	Install .deb on several Ubuntu releases	No dependency errors	Passed after removing libicu
T5	Start on GNOME/Wayland	Window covers the screen	Passed after fullscreen fixes
T6	First request to a sleeping server	Kiosk waits, then shows QR	Passed (60 s timeout)
T7	Network unavailable	Clear error and retry	Passed
T8	Broken roster photo URL	Session continues	Passed
T9	Full flow with the mock pipeline	Status, sample report, thank-you	Passed
T10	Phone scans kiosk QR (live)	Kiosk advances itself	Passed (recorded demo)
T11	Kiosk minimises itself during session	Kiosk returns afterwards	**Failed on Wayland**; rolled back
T12	Kiosk clock set in the past	HTTPS works	Failed; fixed by enabling NTP

Because the real pipeline needs the cameras, motors and meditation application, a small test kit was written that replaces it with a mock script. Two environment variables point the kiosk at the mock and a temporary report folder; no test code is included in the package, so the tested kiosk is exactly the one that ships.

5.3) Challenges Faced and Learnings

Table 11: Main challenges and how they were addressed

Challenge	Resolution
Fullscreen did not stick on Wayland	Resizable window, fullscreen applied in code and re-asserted
Package failed on some Ubuntu versions	Bundle the runtime; drop the ICU dependency
No QR scanner at the kiosk	Kiosk shows the QR; the phone scans it
Integrating another team’s software	Script plus report folder; only PDFs newer than the session
Free server sleeps when idle	Long client timeout and a “waking up” message
Phone camera blocked on plain HTTP	Serve all pages over HTTPS
Kiosk could not restore itself	Roll back; avoid self-managing windows on Wayland

Learnings:

- **Constraints can simplify a design:** the missing scanner led to an easier flow for visitors.
- **Integrate through the narrowest contract:** a script and a folder let two teams move independently.
- **Know the platform’s rules first:** fighting Wayland’s window policy cost two releases.
- **Rolling back is a valid decision:** returning to a known-good baseline was faster than patching.
- **Write things down:** keeping the setup guide updated let the lab install each build without me.

CONCLUSIONS & FUTURE SCOPE

6.1) Conclusions

The internship at the INDReA Lab, IIT Hyderabad, produced the complete software side of the Diya meditation kiosk. The key conclusions are:

1. Hardware-free visitor identification

- **Outcomes:** visitors are identified with nothing but their own phone, in three taps.
- **Significance:** removing a scanner reduced cost and points of failure while improving the experience.

2. Reliable unattended kiosk

- **Outcomes:** a fullscreen application that installs with one command, starts on boot and recovers from crashes.
- **Significance:** the kiosk can run all day in a public space without an attendant.

3. Clean integration across teams

- **Outcomes:** the hardware team’s pipeline runs from the kiosk and its report is shown in the app.
- **Significance:** a minimal contract let both teams evolve their parts without breaking each other.

4. Documented, maintainable system

- **Outcomes:** seven versions delivered, each with an updated setup guide, plus a handoff document.
- **Significance:** the lab can operate, update and extend the kiosk after the internship.

6.2) Future Scope of Work

1. Consent and Privacy Screen

- **Objective:** show what is captured and stored, and obtain consent, before any camera starts.
- **Significance:** the kiosk records people and the roster contains ID numbers, so this is needed before public deployment.

2. Inactivity Reset and Pipeline Timeouts

- **Objective:** return to the welcome screen if a visitor leaves mid-session, and add an overall timeout with an error screen to the pipeline and to the alignment loops.
- **Significance:** a stage that never succeeds can then no longer hang the kiosk.

3. Real Calibration Stages

- **Objective:** replace the placeholder scripts called by run1.sh with the hardware team's motor-control and radar stages.
- **Significance:** the visitor will be physically aligned to the sensors, improving data quality.

4. Face Recognition and Report Delivery

- **Objective:** identify enrolled visitors by face and send the report to them by email.
- **Significance:** removes even the phone step and lets visitors keep their results.

5. Continuous Integration and Persistent Database

- **Objective:** build packages automatically as releases with a single version number, and move the database off the free tier, which deletes data after 30 days.
- **Significance:** safer updates and no loss of visitor data.

References

[1] A. Troelsen and P. Japikse, *Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming*, 11th ed. Berkeley, CA, USA: Apress, 2022.

[2] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.

[3] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O’Reilly Media, 2017.

[4] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.

[5] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA, USA: Addison-Wesley, 2010.

[6] D. A. Norman, *The Design of Everyday Things*, revised ed. New York, NY, USA: Basic Books, 2013.

[7] B. Shneiderman, C. Plaisant, M. Cohen, S. Jacobs, N. Elmqvist and N. Diakopoulos, *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, 6th ed. Boston, MA, USA: Pearson, 2016.

[8] H. Brignull and Y. Rogers, “Enticing people to interact with large public displays in public spaces,” in *Proc. IFIP INTERACT*, Zurich, Switzerland, 2003, pp. 17–24.

[9] P. Peltonen *et al.*, “It’s Mine, Don’t Touch!: Interactions at a large multi-touch display in a city centre,” in *Proc. ACM CHI Conf. Human Factors in Computing Systems*, Florence, Italy, 2008, pp. 1285–1294.

[10] J. Kabat-Zinn, *Full Catastrophe Living: Using the Wisdom of Your Body and Mind to Face Stress, Pain, and Illness*. New York, NY, USA: Delacorte Press, 1990.

[11] *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*, ISO/IEC 18004:2015, 2015.

[12] Avalonia UI, “Avalonia documentation,” 2026. [Online]. Available: https://docs.avaloniaui.net/

[13] Microsoft, “.NET application publishing overview,” 2026. [Online]. Available: https://learn.microsoft.com/dotnet/core/deploying/

[14] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2026. [Online]. Available: https://www.postgresql.org/docs/

[15] freedesktop.org, “systemd.service — Service unit configuration,” 2026. [Online]. Available: https://www.freedesktop.org/software/systemd/man/systemd.service.html

[16] K. Høgsberg *et al.*, “The Wayland protocol,” 2026. [Online]. Available: https://wayland.freedesktop.org/docs/html/

[17] Intel Corporation, *Intel RealSense Product Family D400 Series Datasheet*, rev. 016, 2023.

[18] Hi-Link Electronic, *HLK-LD2410 Human Presence Sensing Module: User Manual and Serial Protocol*, 2022.

Annexures

Annexure A: Weekly Work Log

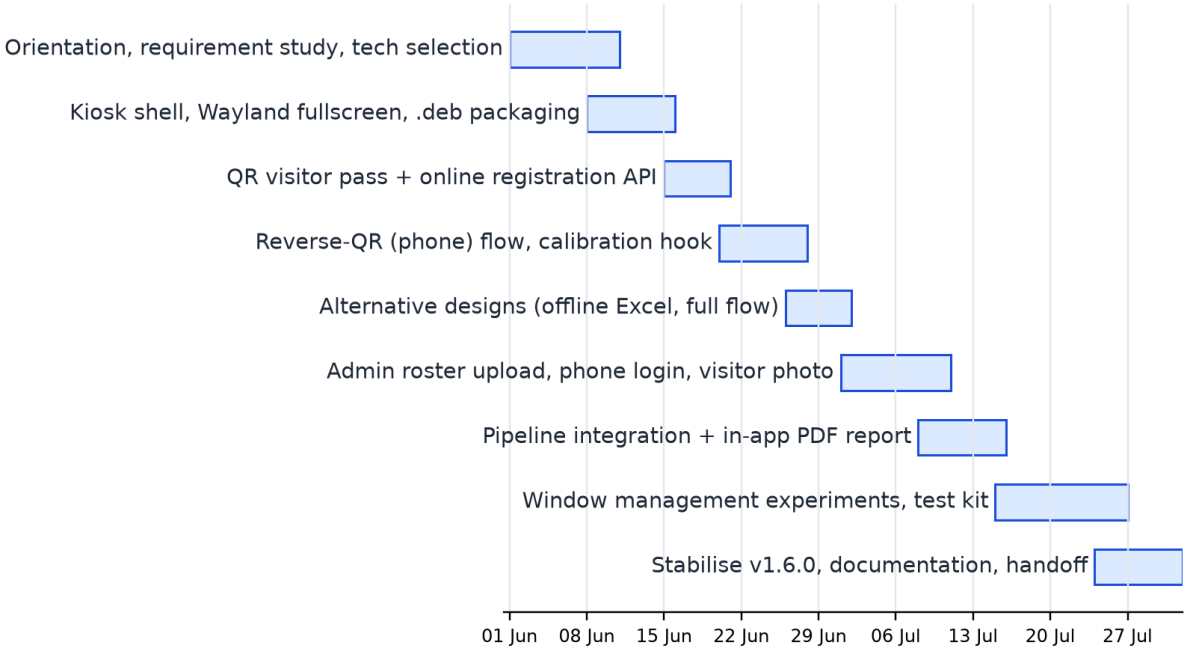


Figure 22: Gantt chart of the internship (1 June – 31 July 2026)

Table 12: Weekly work log

Week	Dates	Work done
1	1–7 Jun	Orientation at the lab; study of the kiosk concept; requirements; choice of .NET 8 and Avalonia
2	8–14 Jun	First kiosk app; two-panel home screen; Wayland fullscreen fixes; .deb builder; setup guide
3	15–21 Jun	Offline install; QR visitor pass (v1.1.0); hosted API with lookup by ID (v1.2.0)
4	22–28 Jun	Reverse QR flow with phone registration (v1.3.0); calibration launch (v1.4.0)
5	29 Jun – 5 Jul	Alternative branches; project overview and FAQ; admin roster and reverse-scan login
6	6–12 Jul	Visitor photo; setup docs; login runs the pipeline and shows the PDF (v1.5.0)
7	13–19 Jul	Window behaviour changes requested by the lab; testing on the kiosk
8	20–26 Jul	Codebase overview; window hand-off experiment; no-camera test kit
9	27–31 Jul	Rollback to the stable baseline; report-ready flow and thank-you (v1.6.0); handoff

Annexure B: Code Snippets

B.1 Choosing the report for the current visitor (ReportRenderer.cs):

```
var newest = new DirectoryInfo(ReportDir)
    .EnumerateFiles("*.pdf")
```

```
    .Where(f => f.LastWriteTimeUtc >= sinceUtc)   // only this session's report
    .OrderByDescending(f => f.LastWriteTimeUtc)
    .FirstOrDefault();
```

B.2 Claiming a session only once (server.js):

```
UPDATE sessions
   SET status = 'claimed', visitor_id = $1, claimed_at = now()
 WHERE token = $2 AND status = 'pending';
```

B.3 Retry rules of the pipeline (run1.sh):

```
while true; do
  until run_step HOME1.py  python3.10 HOME1.py;  do sleep 1; done  # retry
  until run_step SHOOT1.py python3.10 SHOOT1.py; do sleep 1; done  # retry
  run_step CHEST1.py python3.10 CHEST1.py || continue  # from HOME1
  run_step EYE1.py   python3.10 EYE1.py   || continue  # from HOME1
  meditation-app; break
done
```

B.4 Auto-restart service (diya-meditation.service):

```
[Service]
ExecStart=/opt/diya-meditation/DiyaMeditation
Restart=always
RestartSec=2
Environment=DIYA_API_BASE=https://diya-registration.onrender.com
```

Annexure C: Screen Wireframes

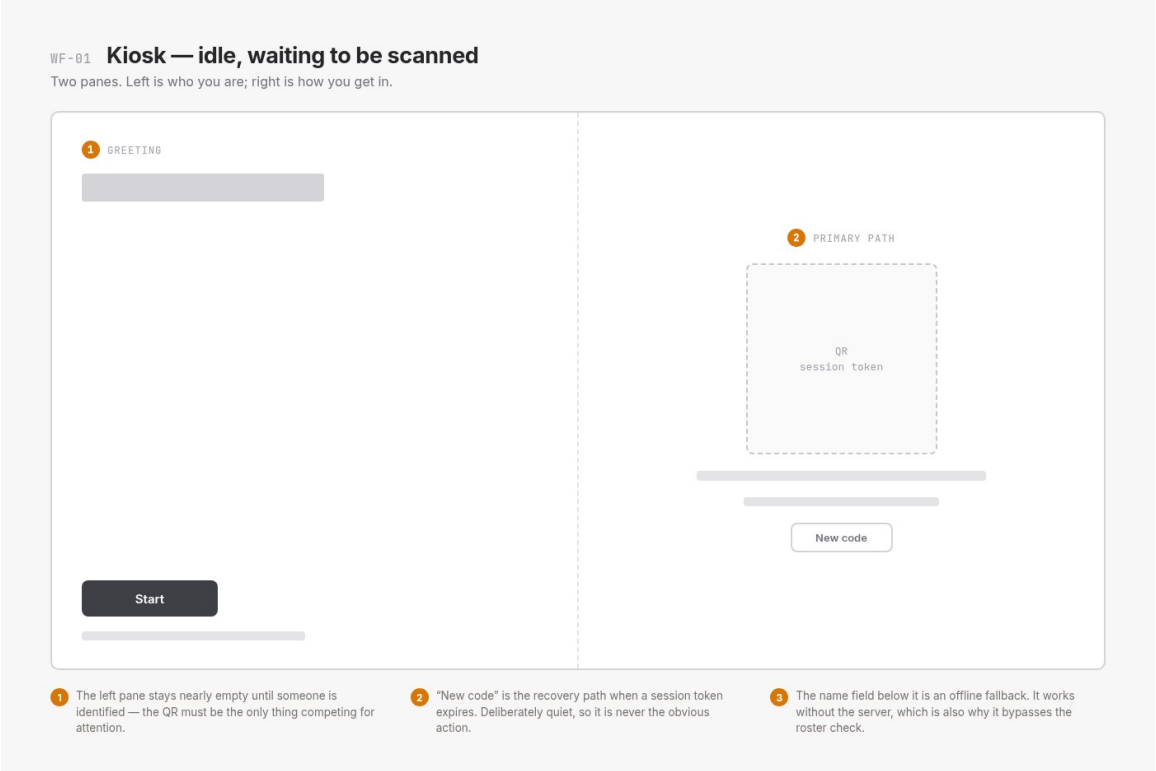


Figure 23: Wireframe WF-01: kiosk idle, waiting to be scanned

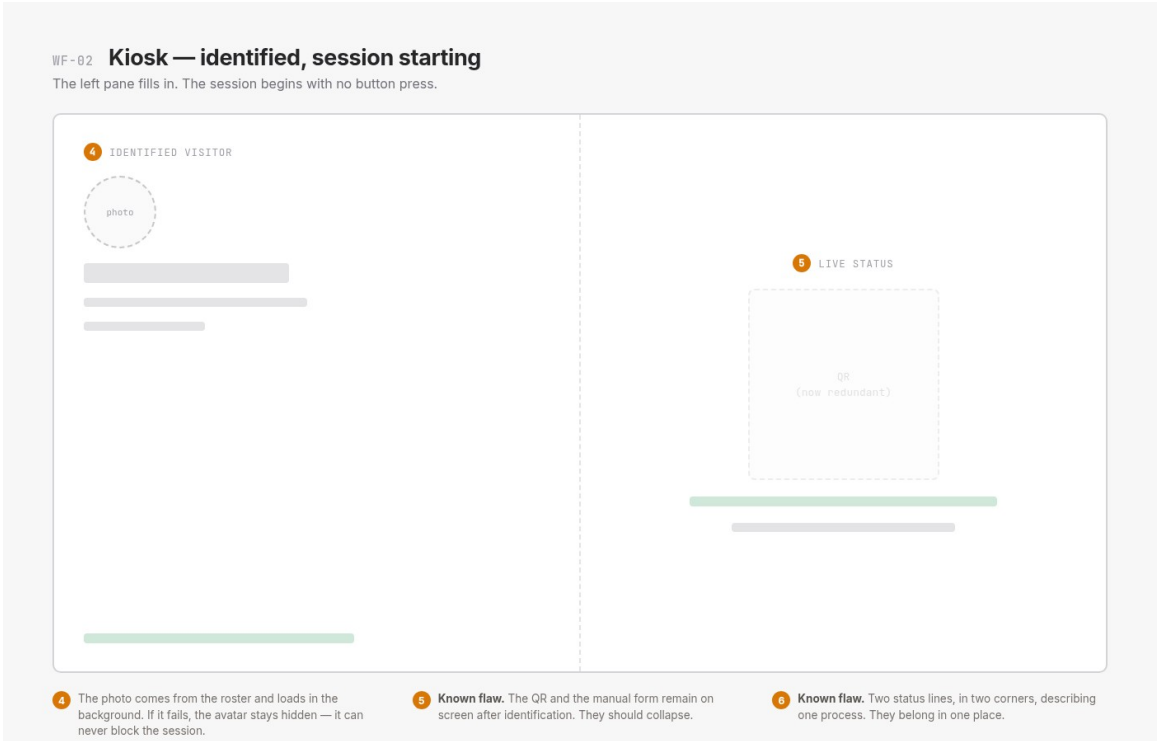


Figure 24: Wireframe WF-02: visitor identified, session starting

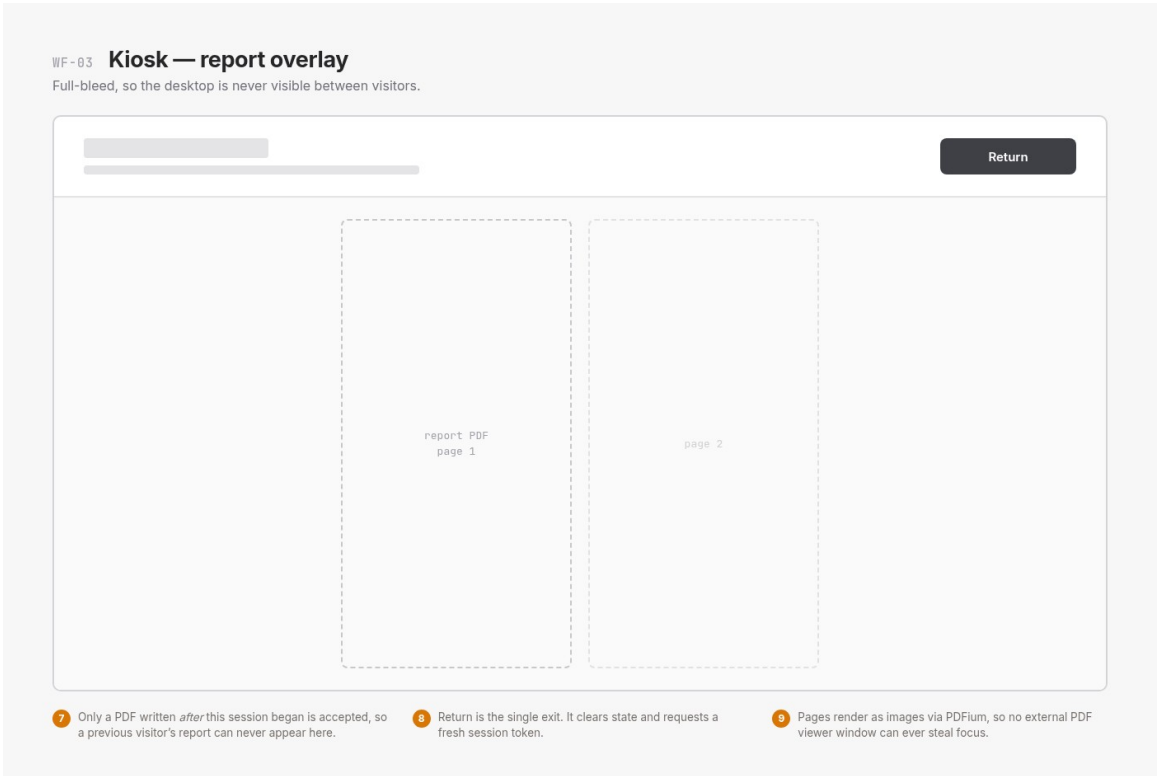


Figure 25: Wireframe WF-03: report overlay

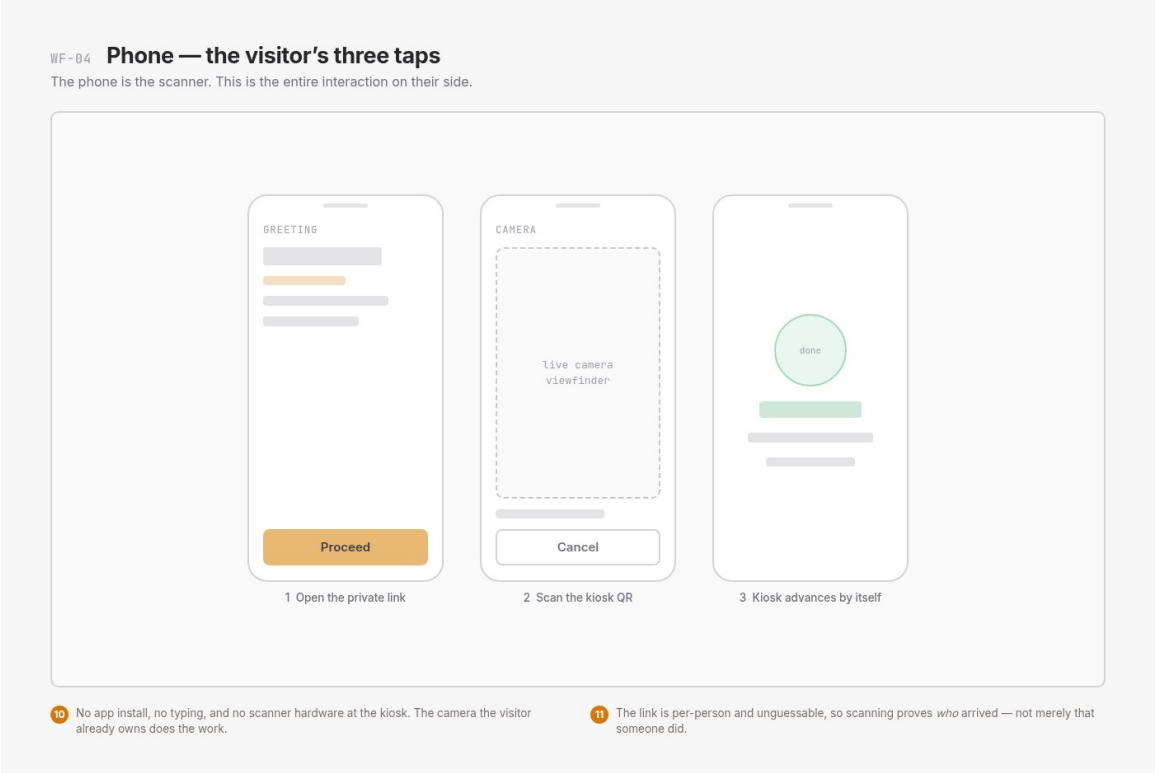


Figure 26: Wireframe WF-04: the visitor's three taps on the phone

Annexure D: Installation Commands

```
systemctl --user stop diya-meditation 2>/dev/null; pkill -f DiyaMeditation
sudo dpkg -i ./diya-meditation_1.6.0_amd64.deb
diya-meditation
curl https://diya-registration.onrender.com/api/health
cd DiyaMeditation && ./deploy/build-deb.sh 1.6.0 amd64
```